

Codio - Monitoring and Analyzing ChatGPT API Usage and Performance

CodioProjectFileEditFindViewToolsEducationHelpNode versionProject Index (static)Configure...

temp.pyx1011121314151617181920212223242526272829303132333435

```
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
```

Guide x

CollapseIntro to GPT-3

Optimize

ChatGPT API, it's important to choose the right model based on your specific needs and requirements. For example, if you need to generate responses quickly and with low latency, you may want to choose a smaller model with fewer parameters. On the other hand, if you need to generate more complex responses or handle a wider range of input types, you may want to choose a larger model with more parameters.

For that lets compare using gpt-4 and gpt-3.5 turbo with the prompt below. Run the code below twice, once with 3.5 and once with 4.. However GPT-4, excels at task requiring reason in comparison to gpt3.5 however, it will take longer to generates response. Please note at the time this course is being written the gpt-4 is not available yet for API calls

```
# Start the timer
start_time = time.time()

# Make the API call to generate the completion
response=openai.ChatCompletion.create(
    model="gpt-4-turbo",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "What's a good book to read on a rainy day? please just name 2."}
    ],
    n=3
)

# Calculate the elapsed time
elapsed_time = time.time() - start_time
print(f"Elapsed time: {elapsed_time:.4f} seconds")
```

TRY IT!

```
Traceback (most recent call last):
  File "temp2.py", line 11, in <module>
    response=openai.ChatCompletion.create(
  File "/home/codio/anaconda3/lib/python3.8/site-
packages/openai/api_resources/chat_completion.py",
line 25, in create
    return super().create(*args, **kwargs)
  File "/home/codio/anaconda3/lib/python3.8/site-
packages/openai/api_resources/abstract/engine_api_resc
line 149, in create
    ) = cls.__prepare_create_request(
  File "/home/codio/anaconda3/lib/python3.8/site-
packages/openai/api_resources/abstract/engine_api_resc
line 106, in __prepare_create_request
    requestor = api_requestor.APIRequestor(
  File "/home/codio/anaconda3/lib/python3.8/site-
packages/openai/api_requestor.py", line 130, in
__init__
    self.api_key = key or util.default_api_key()
  File "/home/codio/anaconda3/lib/python3.8/site-
packages/openai/util.py", line 186, in
default_api_key
    raise openai.error.AuthenticationError(
openai.error.AuthenticationError: No API key
provided. You can set your API key in code using
'openai.api_key = <API-KEY>', or you can set the
environment variable OPENAI_API_KEY=<API-KEY>. If
your API key is stored in a file, you can point the
openai module at it with 'openai.api_key_path =
<PATH>'. You can generate API keys in the OpenAI web
interface. See https://onboard.openai.com for
details, or email support@openai.com if you have any
questions.
```

38% (13:6)

https://codio.com/sandipan-dey/monitoring-and-analyzing-chatgpt-api-usage-and-performance/tree/temp.py

What is one reason to choose a smaller model with fewer parameters when using the ChatGPT API?

☐ Smaller models generate more complex responses.

☐ Smaller models handle a wider range of input types.

☒ Smaller models generally generate responses more quickly and with lower latency.

☐ Smaller models are more accurate at tasks requiring reasoning.

Smaller models with fewer parameters generally generate responses more quickly and with lower latency, making them more suitable for applications where speed and low latency are important considerations.

Check It!

Next

1/1